

面向具身智能的高灵活大规模 强化学习框架Rlinf

于超 | 清华大学深圳国际研究生院 助理教授



于超

助理教授

清华大学深圳国际研究生院

清华-智元机器人联合研究中心主任

- 入选中国电子学会**青年托举工程**项目
- 清华大学**优秀博士毕业生**、**优秀博士论文**、**智能体与多智能体系统优秀博士论文**提名
- 清华大学“**水木学者**”
- 主持**青基**、博后**面上**和**特别资助**项目
- 以第一/通讯作者在高水平国际会议和期刊发表论文 **50** 余篇，包括CVPR Oral、ICML Oral、OSDI、RSS 等，谷歌学术引用 **6000** 余次

科研方向：强化学习、具身智能

多智能体强化
学习算法
MAPPO, 谷
歌学术引用
>3000 次

无人机高并行
强化学习仿真
OmniDrones,
被腾讯开悟
集成

大模型人类反
馈强化学习算
法研究获得
ICML Oral
(top 1.5%)

具身智能大规模强化学习框架 RLinf

- **首个**面向具身智能的高灵活大规模强化学习框架
- 系列工作发表于OSDI（每年收录约50篇）和RSS等
- 开源半年 GitHub Star **超 3300** 次, Fork **450** 余次
- 获得EAI-100 2026年度**十大突破奖**
- 被 NVIDIA Isaac Lab 收录

2022

2023

2024

2025

为什么要做具身智能？

机器人？ 我们不是在重新发明机器人
而是在重新思考“智能如何存在”



具身智能

在物理世界中扩展智能边界

怎么扩展？

真实世界在线策略学习

纸上得来终觉浅

(Offline, 离线数据)

绝知此事要躬行

(Online, 真实在线交互数据)



专家数据



自我尝试 (试错)



自我尝试 (成功)

Offline

Online

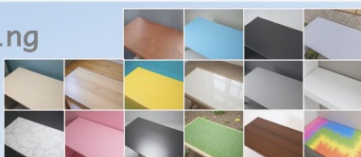
无交互，不理解

Step 1: 机理验证 (25.2 ~ 25.5)

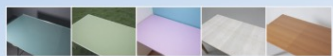
研究了强化学习在三种泛化能力上带来的提升

Vision

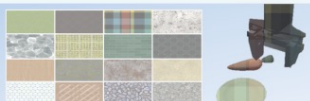
Training



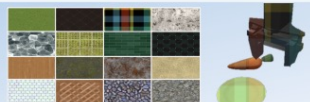
Unseen Table



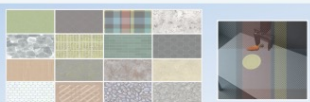
Dynamic Texture (weak)



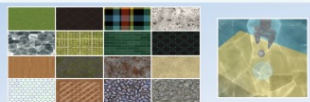
Dynamic Texture (strong)



Dynamic Noise (weak)



Dynamic Noise (strong)



Semantics

Training



Unseen Objects



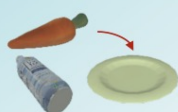
Unseen Receptacles



Unseen Instruction Phrasings

"The \$O\$ should be placed on the \$R\$."

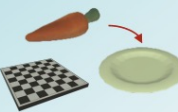
Multi-Object (both seen)



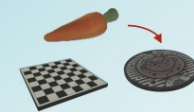
Multi-Object (both unseen)



Distractive Receptacle



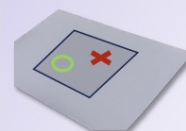
Multi-Receptacle (both unseen)



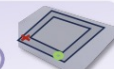
RL Fine-tuning
for VLA
Generalization

Execution

Training



Unseen Position (Object & Receptacle)



Unseen Robot Init Pose



Mid-Episode Object Reposition



视觉泛化
不同材质, 不同背景

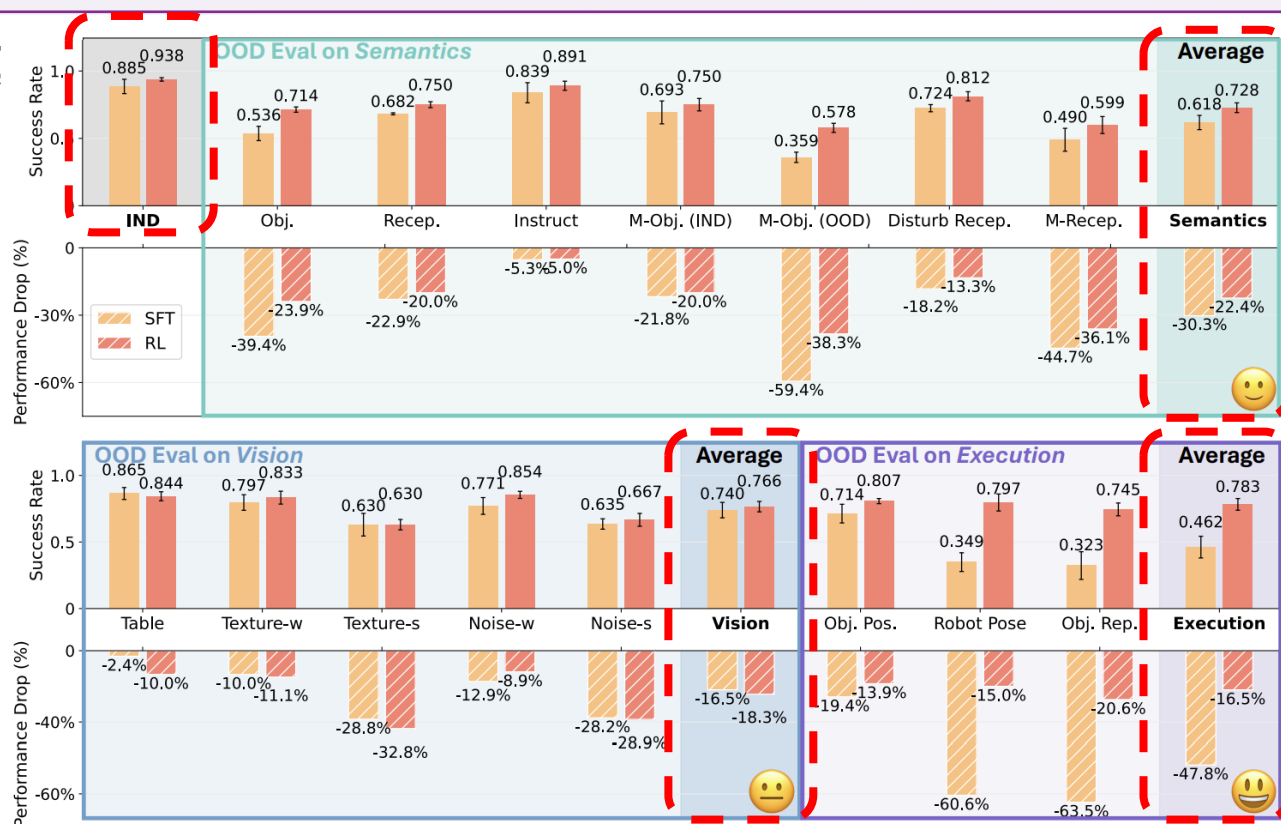
语义泛化
不同物品, 不同指令

执行泛化
不同位置, 不同操作

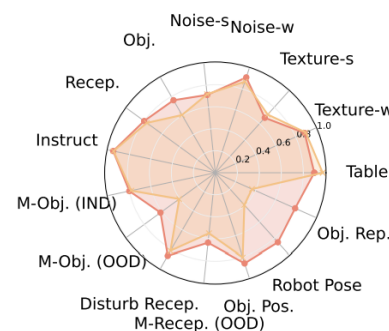
Step 1: 机理验证 (25.2 ~ 25.5)

相比 SFT, 强化学习训练的 VLA 模型在**语义层面和执行层面泛化性提升显著**

SFT 模型和RL 模型在训练集表现相似



在视觉、语义、执行泛化上均有提升，执行层面泛化提升**超过30%**



Step 1.1: 机理可行, 系统拉垮 (25.2 ~ 25.5)

(当时的认知)
最主要的问题是

慢



找一下有没有大规模框架可以用



1周后

很遗憾!
这好像是个全新领域?

Step 1.2: 系统拉垮在哪 (25.5 ~ 25.6)

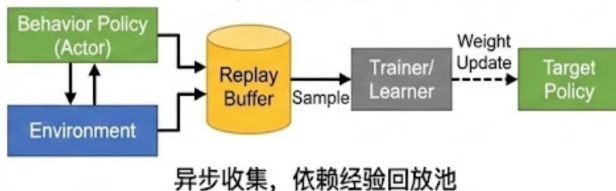
RL训练流程多样、组件高度异构，现有框架调度模式无法满足具身智能需求

不同算法训练流程差异大

1. On-Policy RL (在线策略)



2. Off-Policy RL (离线策略 - 带缓存)

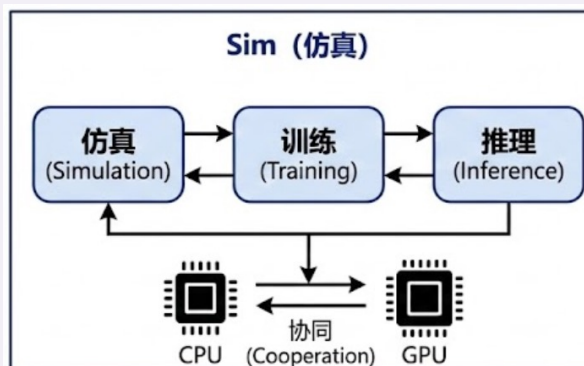


3. Offline RL (全离线/批式强化学习)

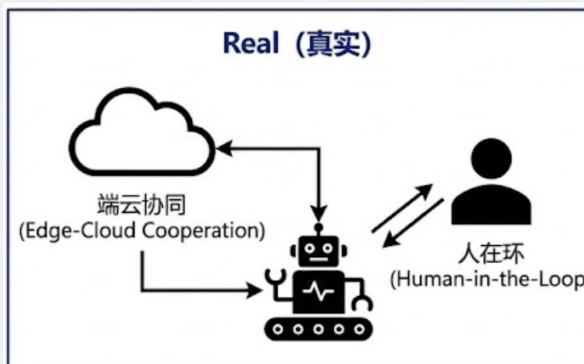


不同组件高度异构

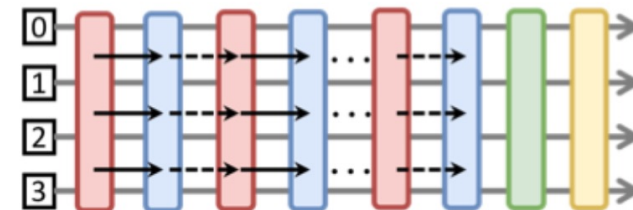
Sim (仿真)



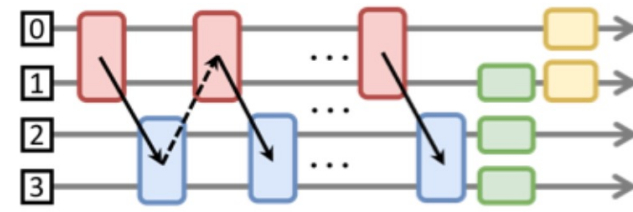
Real (真实)



训练效率低、灵活性差



Collected mode
组件切换开销大



Disaggregated mode
更高的计算资源开销

Step 2: 构造新系统 (25.5 ~ 25.9)

(今天的认知)
具身智能+强化学习:
基础设施是关键



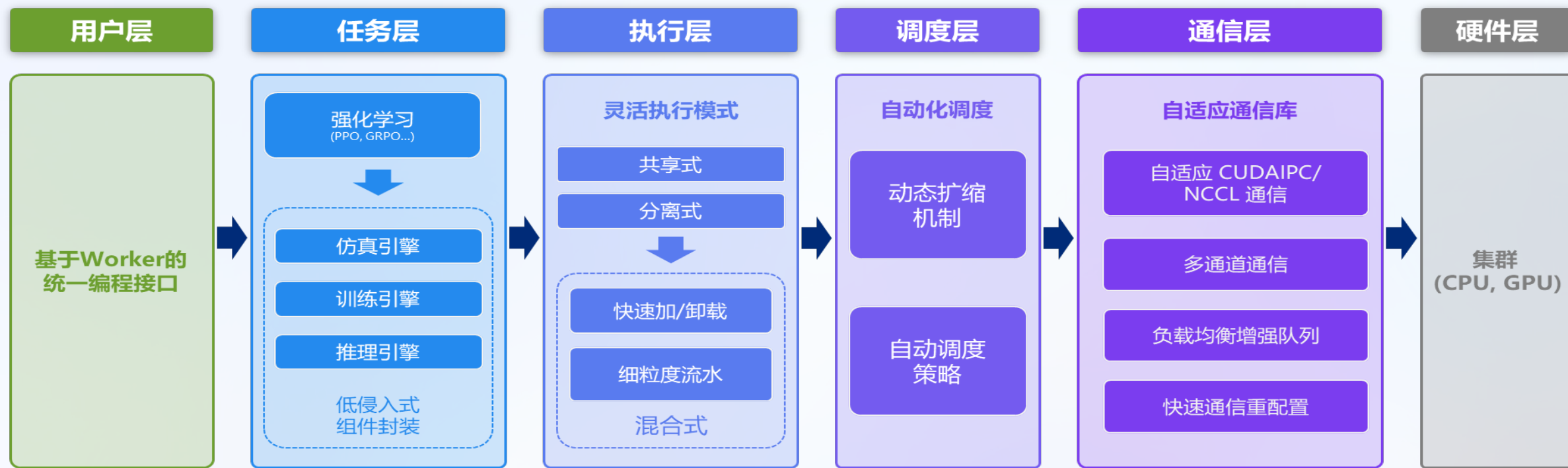
慢只是问题之一，灵活性是社区最大的需求



1周后

打造一个新系统!
来时路: 算法驱动的系统设计!

Step 2: Rlinf: 首个面向具身智能的高灵活大规模强化学习框架



提出新型强化学习编程范式：宏到微流变换 (Macro-to-Micro Flow Transformation, M2Flow)
解耦上层逻辑与底层优化，兼顾灵活性与高效性

Chao Yu, et al., "RLinf: Flexible and Efficient Large-scale Reinforcement Learning via Macro-to-Micro Flow Transformation", in *USENIX Symposium on Operating Systems Design and Implementation (OSDI '26)*.

Step 2.1: 系统得劲了一些，算法又不得劲了（25.9 ~ 25.10）

Large Model

- Knowledge Compression
 - Action Decoder

Large Model

- Slow
- Knowledge Compression

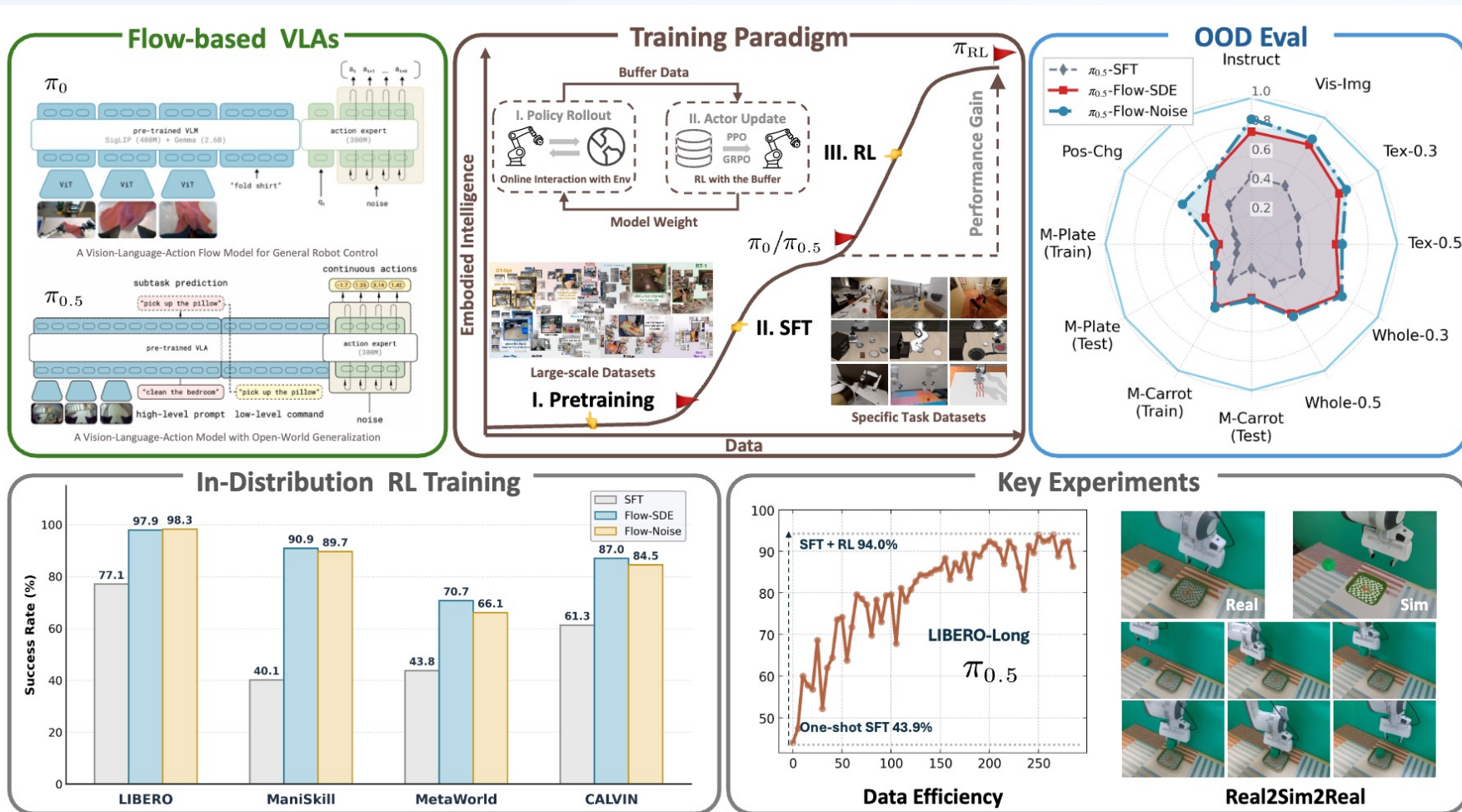
已有工作主要微调自回归VLA模型
流式VLA模型（如 π 系列）无法简单计算
对数似然，难以直接使用RL微调

Action Expert

- Fast
- Interact with Physical World



Step 2.2 : 上算法! π RL 流式VLA模型训练 (25.6 ~ 25.10)



训练三种模型:

π_0 $\pi_{0.5}$ GR00T

提出两种算法:

Flow-Noise、Flow-SDE

四个测试集:

LIBERO、ManiSkill、
MetaWorld、Calvin

取得30% ~ 50%性能提升!

R2S2R实现真机部署测试

Step 2.3: 打磨系统（25.9 至今）

提问：
RLinf系统的高灵活性体现在哪？



For 计算机系统人员，移步RLinf系统详细讲解视频
(1页ppt讲不清楚)



For 具身智能研究人员，打开RLinf Github

RLinf支持矩阵

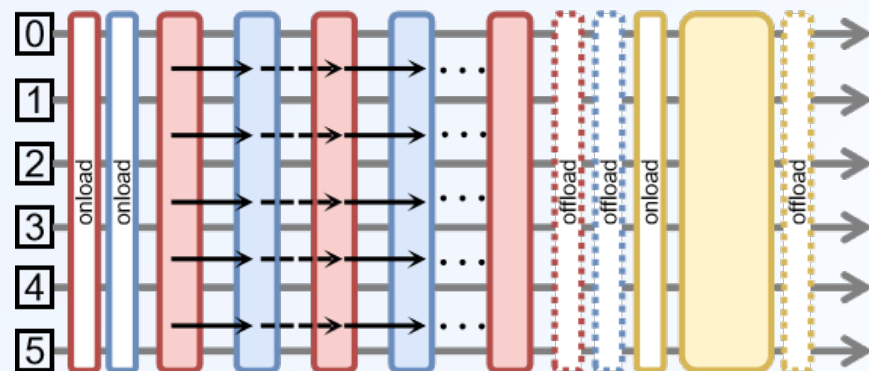
Simulators	Models	Algorithms
• ManiSkill ✓ • LIBERO ✓ • LIBERO-Pro & LIBERO-Plus ✓ • RoboTwin ✓ • RoboVerse ✓ • BEHAVIOR ✓ • MetaWorld ✓ • IsaacLab ✓ • CALVIN ✓ • RoboCasa ✓ • Franka-Sim ✓ • EmbodiChain ✓ • More...	• VLA ◦ π_0 ✓ ◦ $\pi_{0.5}$ ✓ ◦ OpenVLA ✓ ◦ LingBot-VLA ✓ ◦ OpenVLA-OFT ✓ ◦ GR00T ✓ ◦ Dexbotic ✓ ◦ StarVLA ✓ • VLM ◦ Qwen2.5-VL ✓ ◦ Qwen3-VL ✓ • World Model ◦ OpenSora ✓ ◦ Wan ✓ • Custom Models ◦ MLP-Policy ✓ ◦ CNN-Policy ✓ • Reward Model ◦ ResNet ✓ • World Action Model ◦ DreamZero ✓	• RL Algos ◦ IQL ✓ ◦ GRPO ✓ ◦ PPO ✓ ◦ DAPO ✓ ◦ Reinforce++ ✓ ◦ SAC ✓ ◦ CrossQ ✓ ◦ RLPD ✓ ◦ SAC-Flow ✓ ◦ DSRL ✓ ◦ RECAP (CFG) ✓ • SFT ◦ Full-parameter SFT ✓ ◦ LoRA SFT ✓ ◦ VLM SFT ✓ ◦ DAgger ✓ ◦ HG-DAgger ✓

Real-world Robotics	Data Collection
• Franka Arm ◦ Intel RealSense ✓ ◦ Stereolabs ZED ✓ ◦ Franka Hand ✓ ◦ Ruiyan Hand ✓ ◦ Robotiq 2F-85 / 2F-140 ✓ • XSquare Turtle2 ✓ • Dual-franka ✓ • DOS-W1 ✓ • GimArm ✓ • More...	• GELLO ✓ • SpaceMouse ✓ • PICO VR ✓

Single-Agent	Multi-Agent
• SearchR1 ✓ • rStar2 ✓ • AgentLightning Calc-X ✓ • Online Coder ✓ • Math Reasoning RL ✓	• WideSeek-R1 ✓

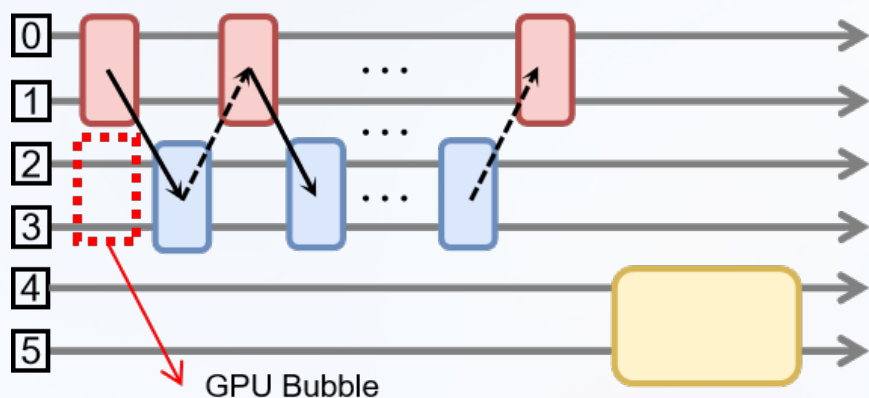
RLinf内置调度模式

共享



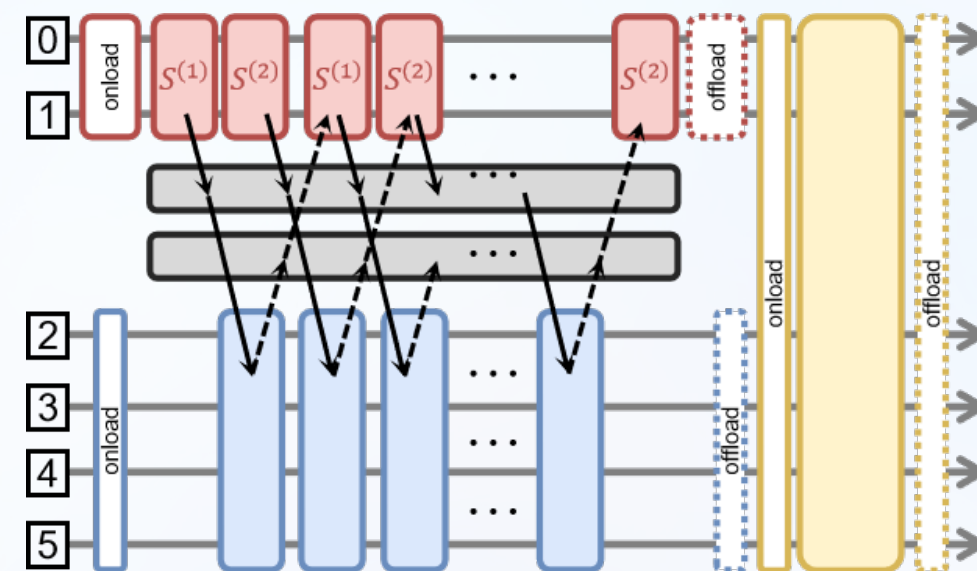
(RLinf CPU仿真强化采用)

分离



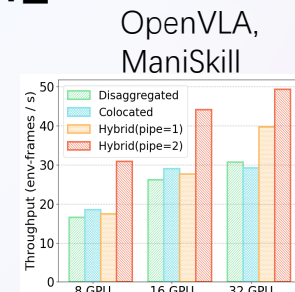
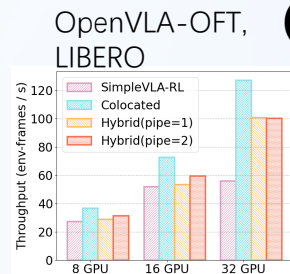
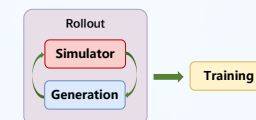
通常要搭配异步算法 (RLinf真机强化采用)

混合



保持同步算法属性的同时拉满系统吞吐

(RLinf GPU仿真强化采用)



Step 3: 还是不对，真实世界在哪里？（25.10）

仿真世界不可弥合的虚实差距，真实世界在线策略学习

（当时的认知）
又要重新做一套系统了？

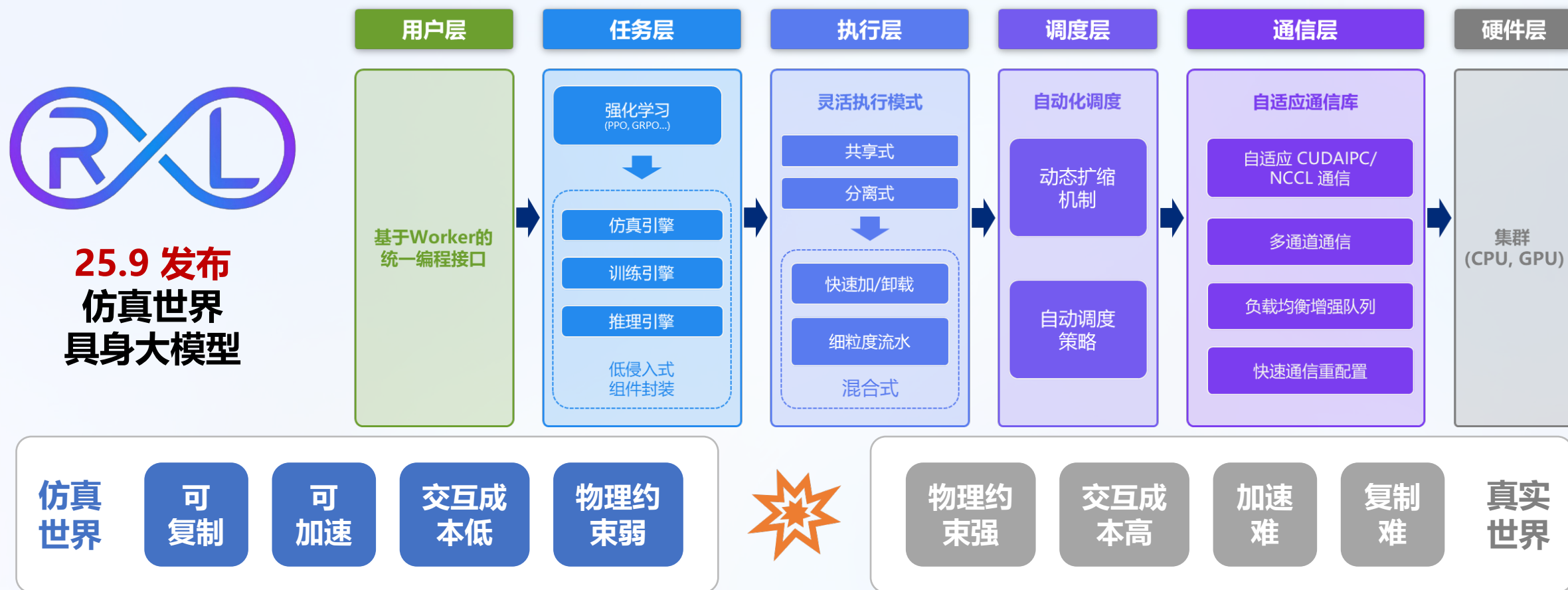


写吧



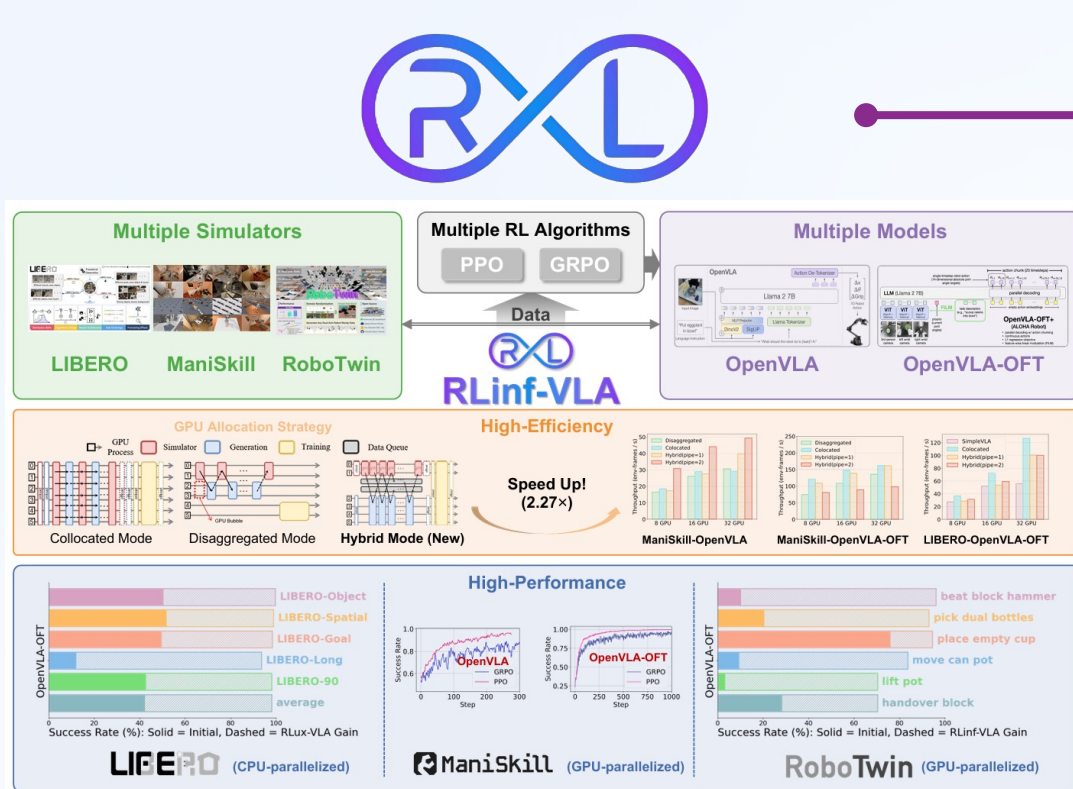
等一下！RLinf的高灵活性设计已经满足了大部分需求！但是确实有新需求，增量开发即可！
（喜大普奔）

Step 3: 真实世界的问题是什么？（25.11 ~ 26.2）

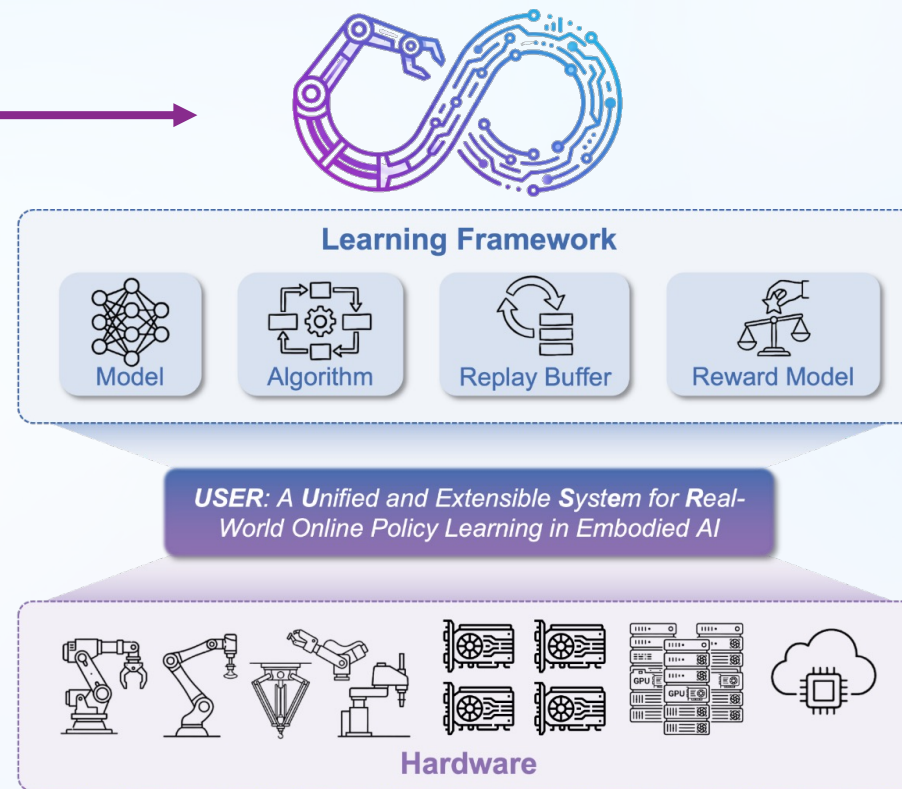


规模化真实世界可扩展的在线策略学习是系统与算法深度耦合的复杂问题

Step 3.1: 升级! USER! (25.11 ~ 26.2)



Scaling 模型

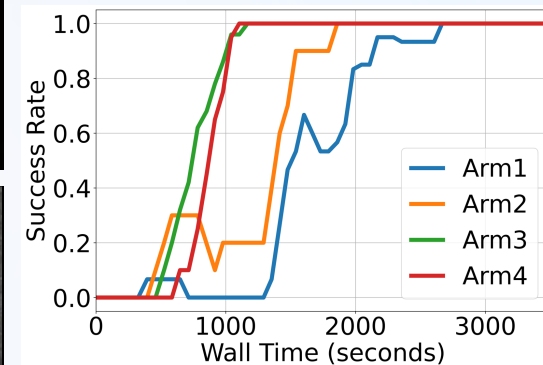
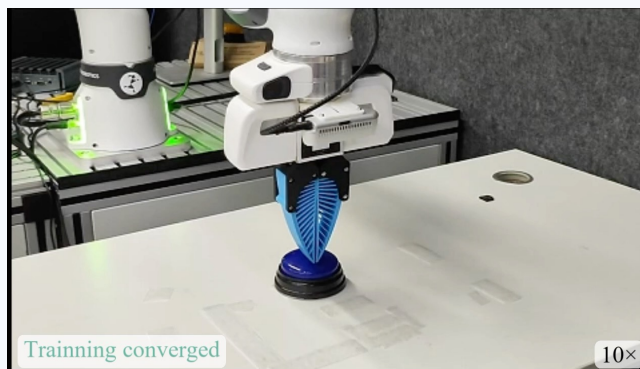
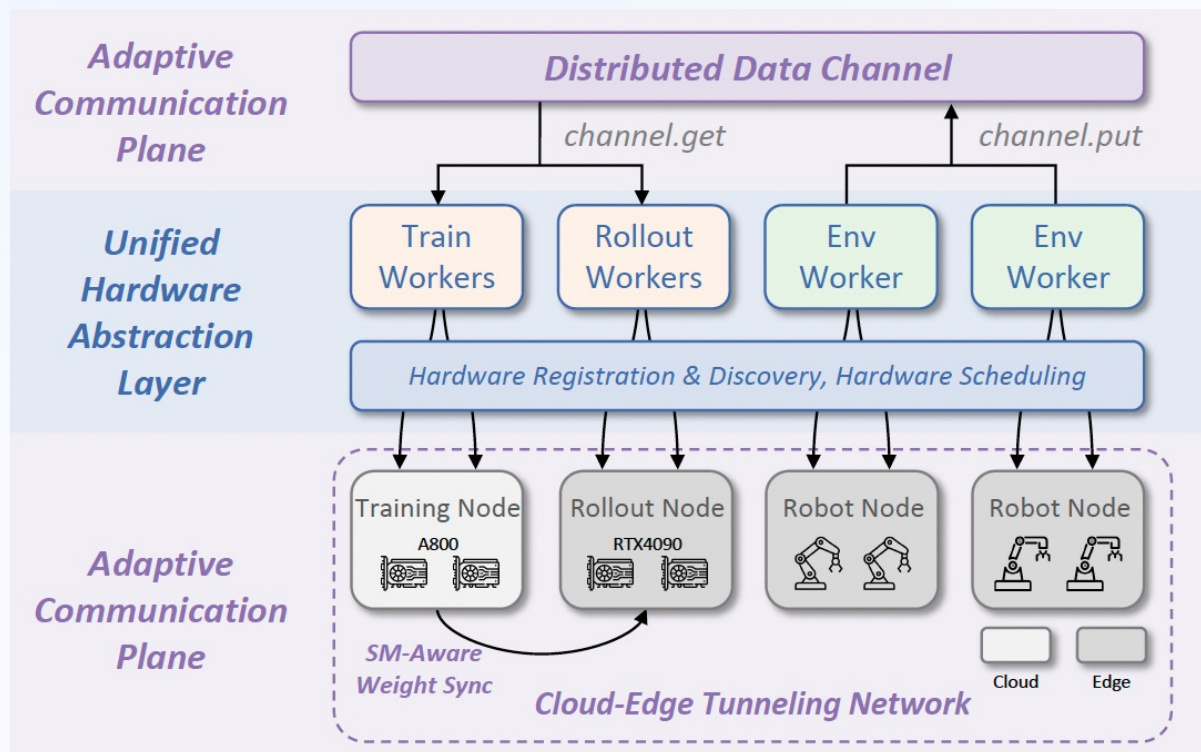


Scaling 模型+真机

从在仿真世界里训练智能，到在真实世界里训练“具身智能”

RLinf-USER（系统层）设计1：统一硬件抽象层

在规模化在线策略学习场景下，存在**大规模异质硬件资源（机器人、端云计算卡）难以统一管理和调度**的问题，提出采用**统一硬件抽象层**，将机器人硬件与计算硬件放在相同层级管理，实现“**像使用GPU一样使用机器人**”，具有高度可扩展性。

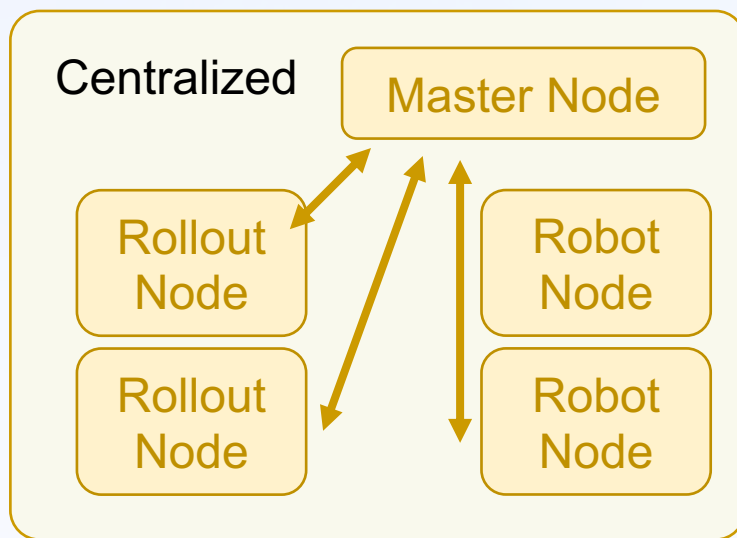


4机跨域训练

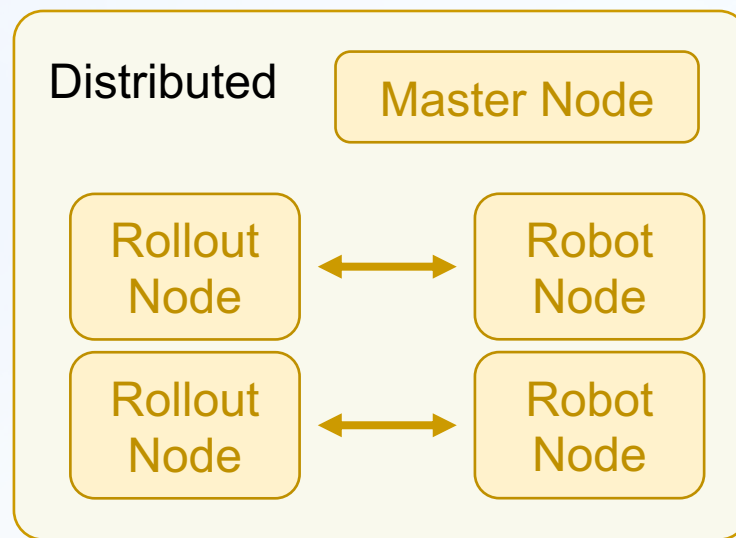
实验：支持多机器人“即插即用”（自注册和自发现），实现异构机器人（Franka、方舟无限）同时在线学习

RLinf-USER（系统层）设计2：自适应通信平面

具身大模型通常需要端云协同训练，面临端云网络跨域互通困难、集中式通信跨域开销大等问题，设计了自适应通信平面，通过网络隧穿和分布式数据通道降低跨域通信负担，实现流量本地化。



集中式数据通道



分布式数据通道

Domain	Distributed	Rollout (s/chunk)	Interact (s/chunk)	Send Obs (s/data)	Send Action (s/data)	Total Generation Time ↓ (s/episode)
cross	w/	0.002(±0.000)	0.106(±0.001)	0.042 (±0.031)	0.070 (±0.017)	21.979 (±0.435)
	w/o	0.002(±0.000)	0.107(±0.001)	0.671(±0.012)	0.270(±0.009)	69.265(±1.905)
same	w/	0.006(±0.001)	0.106(±0.002)	0.025 (±0.006)	0.028 (±0.008)	17.304 (±0.001)
	w/o	0.007(±0.001)	0.106(±0.002)	0.169(±0.018)	0.021(±0.008)	18.696(±0.710)

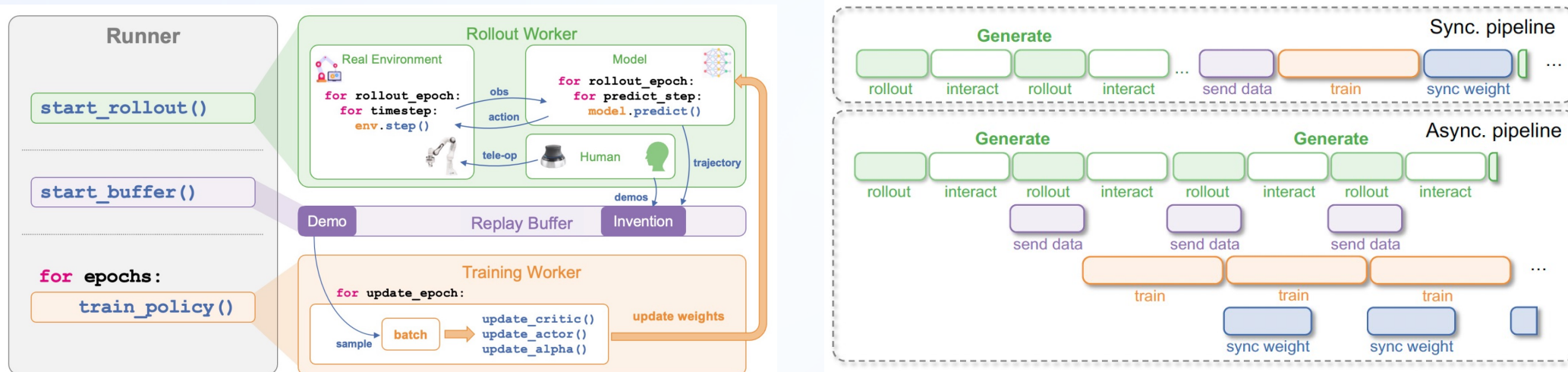
启动程序

实验：实现跨越千里的端云协同训练
(北京-深圳两地多机同时学习)，
跨域场景下学习效率提升约3倍。

跨域及同域网络条件下时间开销

RLinf-USER（算法层）设计3：全异步训练架构

真实世界训练数据效率是第一瓶颈，仿真世界同步训练架构效率低。针对该问题，设计了全异步训练架构，解耦数据生成、传输、训练与权重同步，训练吞吐提升5.7倍。

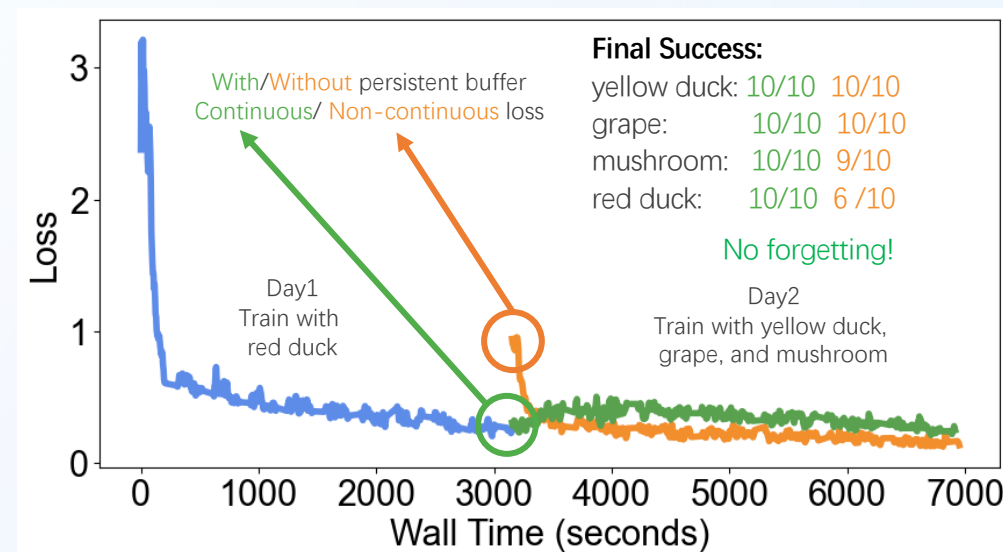
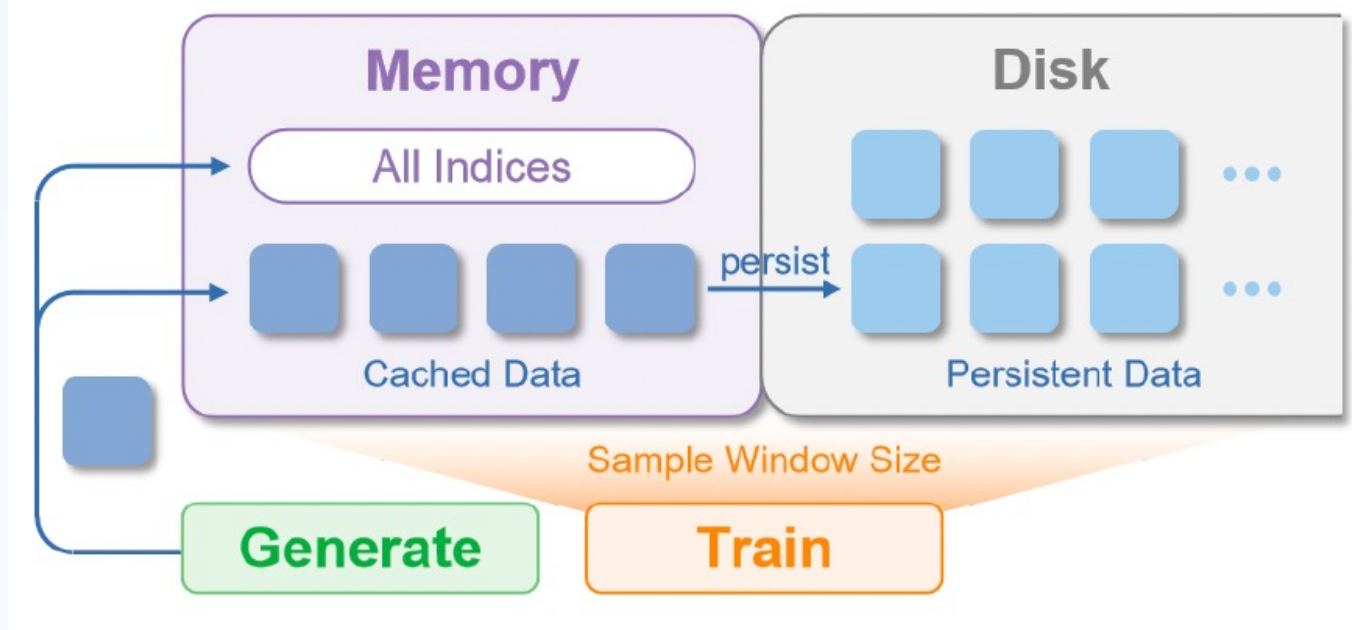


		Rollout (s/chunk)	Interact (s/chunk)	Send data	Train (s/update)	Sync weights	Generation Period ↓ (s/episode)	Training Period ↓ (s/update)
π_0 + HG-Dagger	Sync	0.214(±0.001)	1.093(±0.013)	0.606(±0.028)	6.128(±0.247)	0.816(±0.024)	45.068(±0.304)	45.011(±0.251)
	Async	0.213(±0.016)	1.091(±0.010)	7.903(±)	5.969(±0.196)	0.789(±0.052)	37.538±0.363	7.903(±0.234)
	Speed Up	/	/	/	/	/	1.20×	5.70×
CNN + SAC	Sync	0.006(±0.002)	0.108(±0.002)	0.144(±0.008)	0.108(±0.002)	0.162(±0.004)	20.291(±0.632)	0.643(±2.984)
	Async	0.004(±0.004)	0.107(±0.002)	0.004(±0.001)	0.123(±0.005)	0.174(±0.003)	13.108(±0.218)	0.135(±0.034)
	Speed Up	/	/	/	/	/	1.55×	4.61×

同步架构和异步架构训练吞吐对比

RLinf-USER（算法层）设计4：持久化缓存感知缓冲区

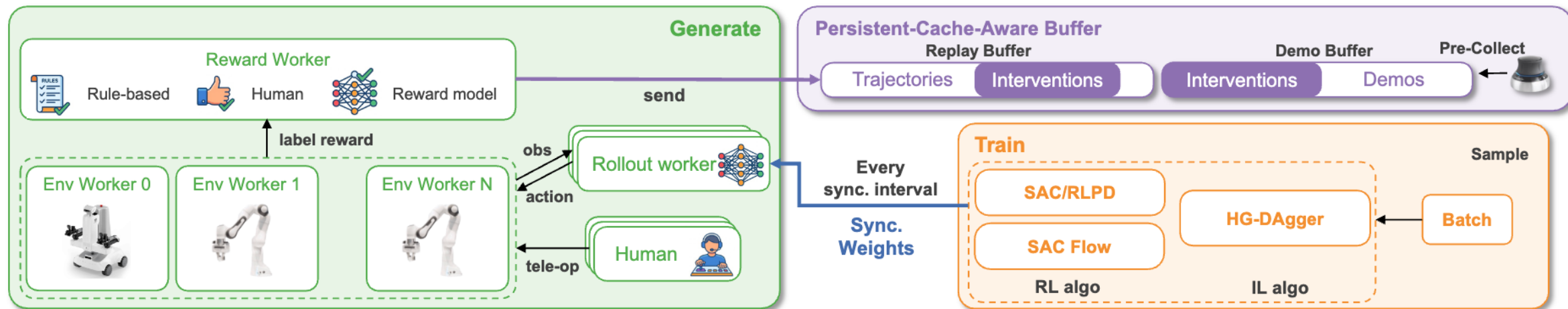
长周期在线学习中多模态数据体量持续增长、纯内存缓冲区容量受限、纯磁盘存储难以满足高频采样需求。针对该问题，设计了持久化缓存感知缓冲区。通过内存缓存与磁盘持久化存储协同，结合动态索引组织与异步存取机制，实现了海量历史数据的高效采样与持续复用。



实现：在面向 TB 级数据规模的在线学习场景中实现了接近零等待的数据访问体验。

支持断点续训、低遗忘的跨日多任务持久化训练

RLinf-USER (算法层) 设计4: 成果展示

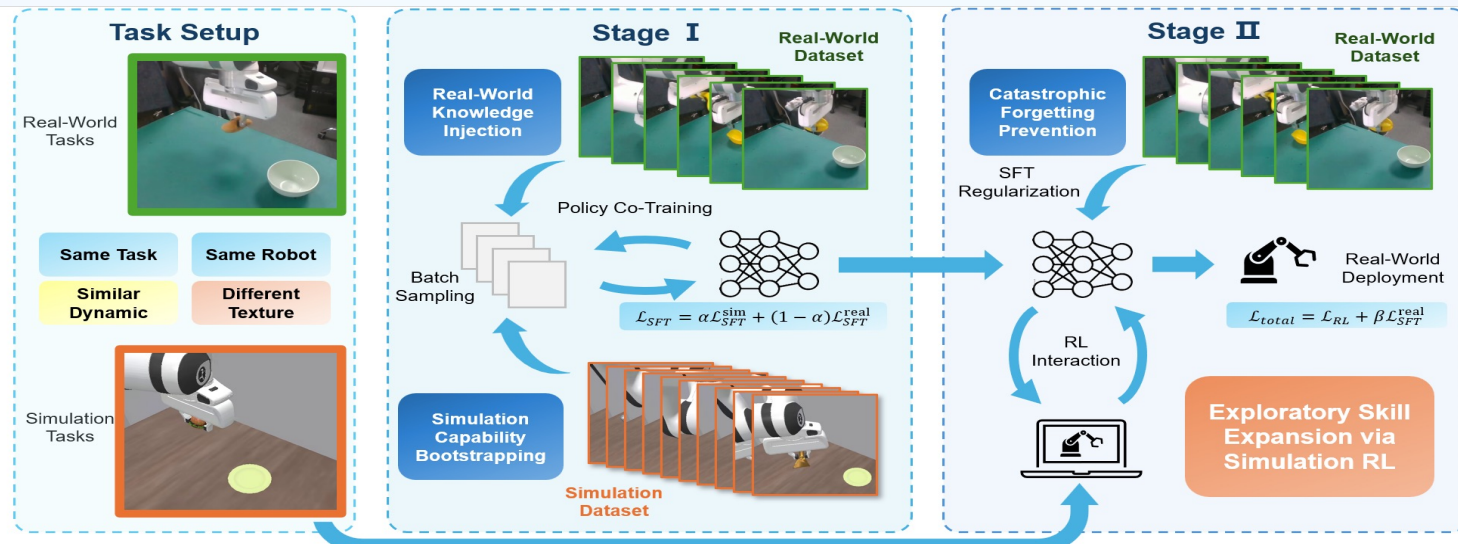
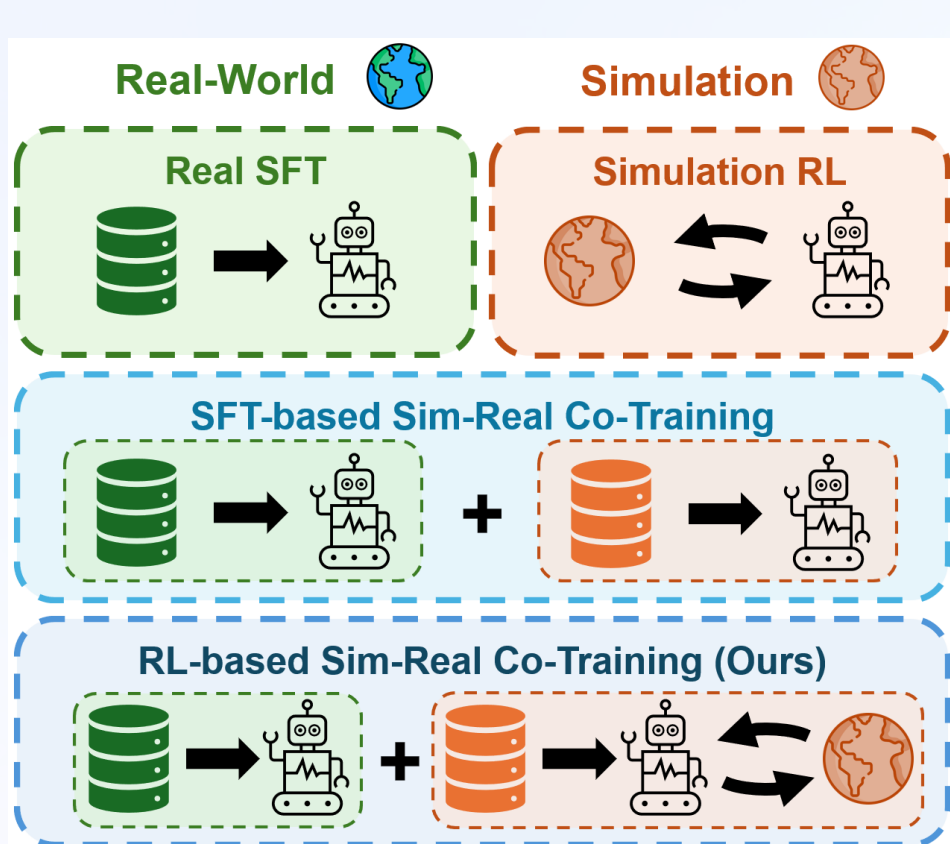


Task Name	Model	Algorithm	Reward Type	Demo (trajs)
Peg Insertion	CNN	SAC	rule-based, dense	/
	Flow	RLPD	rule-based, sparse	20
		SAC Flow		
Charger	CNN	SAC	rule-based, dense	/
	Flow	RLPD	rule-based, sparse	20
		SAC Flow	rule-based, sparse	
Cap Tightening	CNN	RLPD	human reward, sparse	20
Pick-and-Place	CNN	RLPD	human-reward, sparse	40
	π_0	HG-Dagger	/	40
Table Clean-up	π_0	HG-Dagger	/	40

TABLE II: USER enables significant performance gains after online training for foundation VLA models.

	Before online training	After online training
Pick-and-Place	39/60	58/60
Table Clean-up	9/20	16/20

Step 3.2 : 仿真怎么用? RL-Co支持仿真和真机联合训练



两阶段协同支持仿真和真机联合训练

任务设定: 构建“数字表亲”仿真环境

阶段一: 基于SFT的仿真-真实协同训练初始化

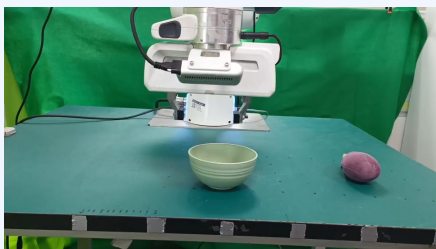
阶段二: 基于强化学习的仿真-真实协同训练

特性一: 真实和仿真环境均无需大量真实世界数据, 无需高保真度仿真器

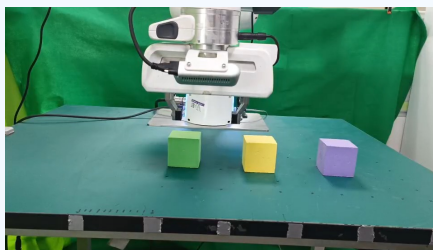
特性二: SFT和RL均能够高效学习初始策略, 扩展模型的能力边界

Step 3.2 : 仿真怎么用? RL-Co支持仿真和真机联合训练

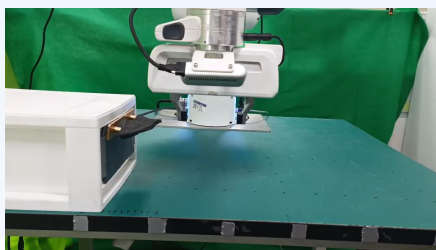
Pick and Place



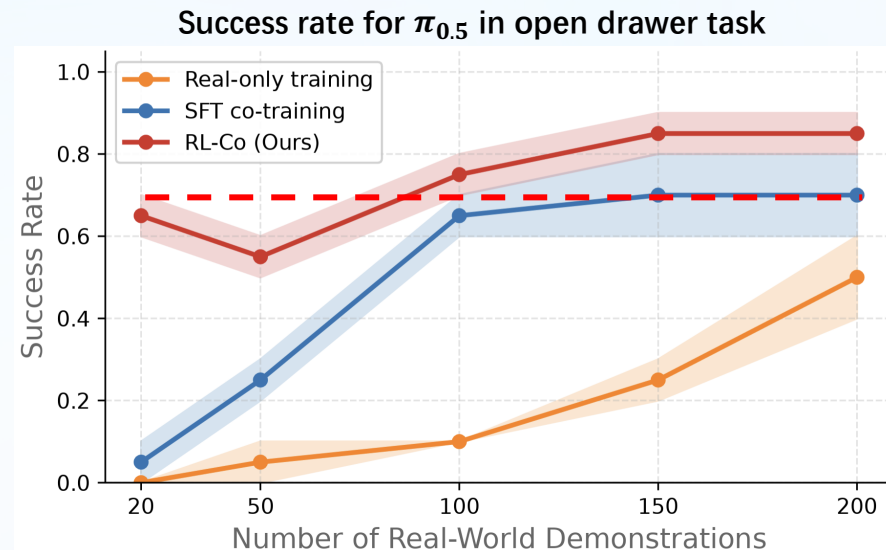
Push Cube



Open Drawer



Close Drawer



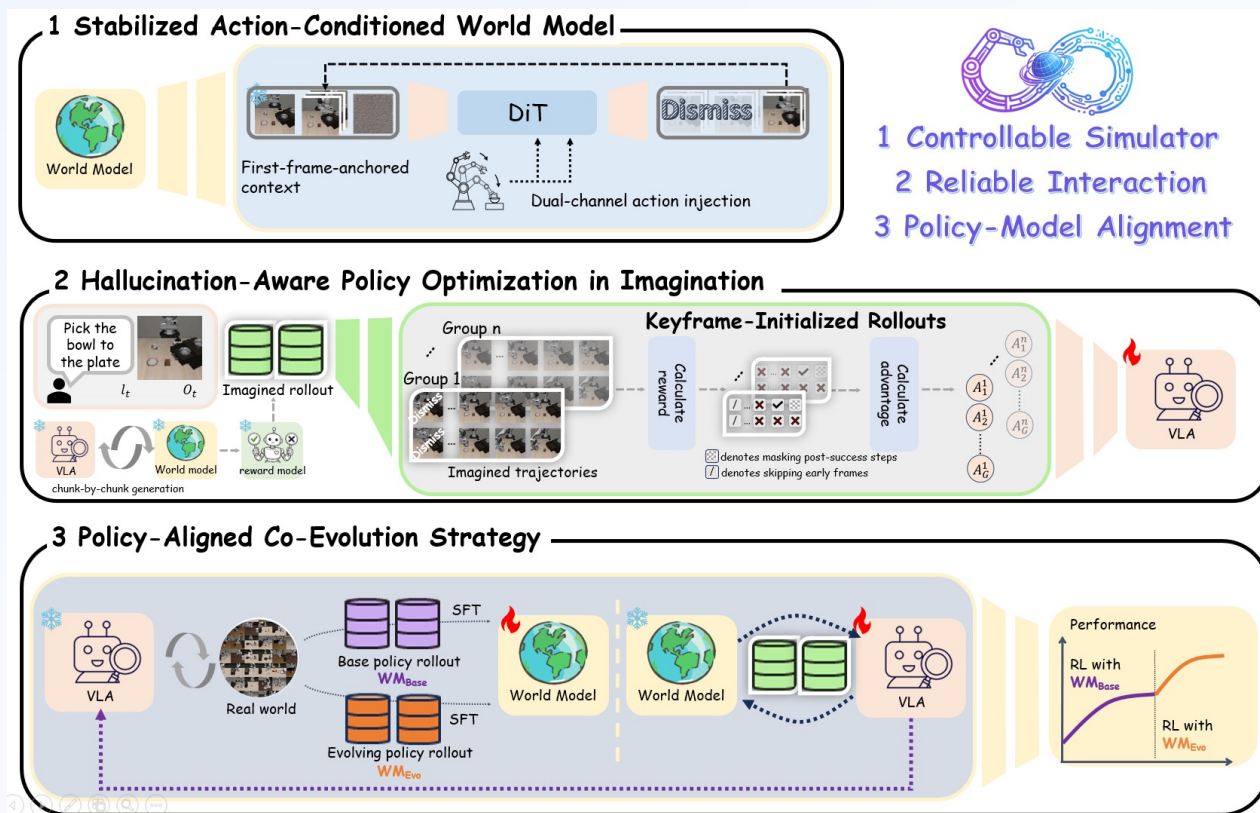
VLA Model	Experiment Setting	Pick and Place	Push Cube	Open Drawer	Close Drawer	Avg
OpenVLA	Real-Only Training	6.3 $\pm$ 0.0	20.0 $\pm$ 13.3	0.0 $\pm$ 0.0	10.0 $\pm$ 10.0	16.5 $\pm$ 13.3
	SFT Co-Training	23.4 $\pm$ 4.7	51.7 $\pm$ 5.0	0.0 $\pm$ 0.0	85.0 $\pm$ 5.0	40.0 $\pm$ 3.7
	RL-Co (Ours)	58.8 $\pm$ 10.0	68.3 $\pm$ 11.7	35.0 $\pm$ 15.0	95.0 $\pm$ 5.0	64.0 $\pm$ 0.7
$\pi_{0.5}$	Real-Only Training	71.9 $\pm$ 9.4	0.0 $\pm$ 0.0	0.0 $\pm$ 0.0	35.0 $\pm$ 15.0	26.7 $\pm$ 1.4
	SFT Co-Training	68.8 $\pm$ 9.4	10.0 $\pm$ 3.3	10.0 $\pm$ 0.0	95.0 $\pm$ 5.0	45.9 $\pm$ 4.4
	RL-Co (Ours)	81.3 $\pm$ 9.4	18.4 $\pm$ 1.7	65.0 $\pm$ 5.0	100.0 $\pm$ 0.0	66.2 $\pm$ 4.0

相比 SFT 方法成功率提升
超过 20%

20 条真机数据使用RL-Co训练效果超过 200 条真机数据 SFT

Step 3.3: 真机代价? WoVR支持在想象中训练

从三个相互关联的层面约束世界模型闭环自回归执行所产生的“幻觉”，为强化学习提供优质的训练环境



仿真层：让世界模型稳定且可控

交互层：无需仅基于初始帧“想象”，采用关键帧初始化技术

对齐层：策略和模拟器共同进化

实验测试：在世界模型性能指标、仿真环境性能、真机实验三类实验测试下 WoVR相比于基线模型和算法均有提升

Step 3.4: 真机代价? WoVR支持在想象中训练

从三个相互关联的层面约束世界模型闭环自回归执行所产生的“幻觉”，为强化学习提供优质的训练环境

① 世界模型性能指标:

WoVR在推理交互的视觉性能超越Cosmos-Predict2和OpenSora

Table 1: World model quality, motion consistency, and efficiency comparison. Rollout denotes the rollout horizon length.

Method	Rollout	FPS ↑	Metrics			
			LPIPS [54]↓	FID [55]↓	FVD [56]↓	FloLPIPS [57]↓
EVAC [42]	512	2.7	0.146	46.528	345.818	0.205
	256		0.130	49.153	354.983	0.192
	128		0.106	44.337	423.132	0.166
Cosmos-Predict2 [58]	512	3.50	0.315	165.862	275.737	0.265
	256		0.226	106.324	203.853	0.306
	128		0.164	77.555	304.456	0.281
OpenSora [22]	512	7.00	0.105	38.478	89.391	0.156
	256		0.082	33.577	94.998	0.122
	128		0.069	33.413	111.643	0.113
WoVR (Ours)	512	23.0	0.091	34.252	68.011	0.154
	256		0.063	24.378	50.041	0.102
	128		0.047	18.553	39.047	0.079

③ 真机实验:

在引入少量轨迹，无需在线交互、只在世界模型中训练策略的情况下，WoVR实现了策略性能提升。

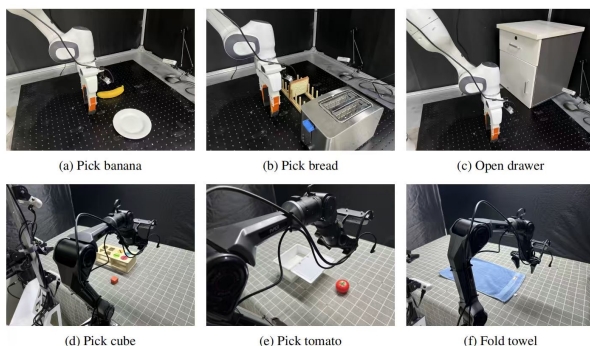


Table 3: Real-world success rates (% , 30 trials per task) on two robotic platforms. We report results on Franka Emika Panda and AgileX Piper. Improvements are shown in parentheses relative to the base policy.

Method	Task 1	Task 2	Task 3	Avg
Franka Emika Panda				
OpenVLA-OFT-base	36.7	70.0	46.7	51.1
WoVR (Ours)	86.7 (+50.0)	90.0 (+20.0)	63.3 (+16.6)	80.0 (+28.9)
AgileX Piper				
OpenVLA-OFT-base	23.3	10.0	13.3	15.6
WoVR (Ours)	33.3 (+10.0)	20.0 (+10.0)	33.3 (+20.0)	28.9 (+13.3)

Table 4: Real-world success rates (% , 30 trials per task) on AgileX Piper using $\pi_{0.5}$ as the VLA backbone. The policy is fine-tuned using Flow-SDE following π_{RL} . Improvements are shown relative to the base policy.

Method	Fold Towel	Pick Cube	Pick Tomato	Avg
$\pi_{0.5}$ -base	20.0	60.0	23.3	34.4
Flow-SDE + WoVR	30.0 (+10.0)	86.7 (+26.7)	53.3 (+30.0)	56.7 (+22.3)

Step 4: RLinf 生态

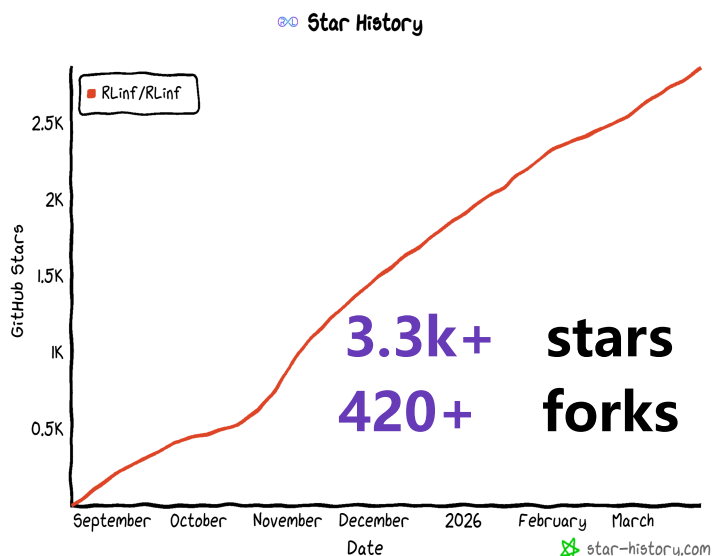
 [Quickstart](#) [Tutorials](#) [Example Gallery](#) [Blog](#) [APIs](#) [FAQs](#)

 Search 

Choose version 

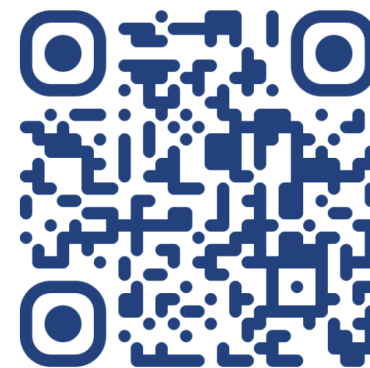


 Ask AI



Welcome to **RLinf**!

RLinf 系列工作全开源



<https://github.com/RLinf/RLinf>
<https://rlinf.readthedocs.io/en/latest>

致力于打造下一代具身智能训练基础设施

Step 4: RLinf用户场景

X SQUARE ROBOT
自变量机器人

HUAWEI

DEXMAL

NVIDIA

AGIBOT

跨维智能
DEXFORCE

PsiBot

引望

摩尔线程
MOORE THREADS

地瓜机器人

Gigami

AMD

产业落地
英伟达用RLinf完成微
创手术器械组装任务



GROOT + online RL
post training
Trocar assemble task

算法开发
极佳科技在RLinf上开
发 $\pi_{\text{step-NFT}}$

π -StepNFT: Wider Space Needs Finer Steps in Online RL for Flow-based VLAs

Siting Wang^{1,2,3}, Xiaofeng Wang^{1,4}, Zheng Zhu^{1,2}, Minnan Pei^{1,2}, Xinyu Cui^{1,2,3,5},
Cheng Dong¹, Jian Zhao¹, Guan Huang¹, Haifeng Zhang^{1,2}, Jun Wang^{1,2}

Abstract

Flow-based vision-language-action (VLA) models excel in embodied control but suffer from intractable likelihoods during multi-step sampling, hindering online reinforcement learning. We propose π -StepNFT (Step-wise Negative-aware Fine-Tuning), a critic-and-likelihood-free framework that requires only a single forward pass per optimization step and eliminates auxiliary value networks. We identify that wider exploration spaces necessitate finer-grained, step-wise guidance for alignment. Empirically, π -StepNFT

A recent systematic analysis (Pan et al., 2025) suggests that after supervised fine-tuning (SFT) converges, the output of flow-based VLAs often collapses to a single mode. Consequently, their efficacy stems from a mechanism of stochasticity injection combined with supervised iterative correction. Injecting noise during training and iterative correction during inference enable the policy to resist error accumulation and adhere closely to the expert manifold during closed-loop execution. However, we contend that relying heavily on the density of expert demonstrations, SFT merely establishes a foundational behavioral manifold that resembles a narrow line, where the model often lacks the

模型部署
地瓜机器人平台支持
RLinf框架



模型后训练
原力灵机Dexbotic模
型使用RLinf框架



与英伟达、智元、自变量、原力灵机、灵初等众多工业界伙伴达成合作

共同构建下一代具身智能基础设施

谢谢
Thanks!